

THIRUVENKADAPRASATH.S

192312016

11	Write a program for the task of Credit Score Classification
----	---

CSV FILE

Income,Age,LoanAmount,CreditHistory,CreditScore

50000,35,15000,1,Good

20000,22,10000,0,Poor

45000,40,20000,1,Good

18000,25,12000,0,Poor

80000,50,30000,1,Good

22000,28,15000,0,Poor

60000,32,18000,1,Good

15000,21,8000,0,Poor

75000,45,25000,1,Good

30000,30,14000,0,Poor

PROGRAM

```
import numpy as np
```

```
import pandas as pd
```

```
class Node:
```

```
    def __init__(  
        self, feature=None, threshold=None, left=None, right=None, value=None  
    ):  
        self.feature = feature  
        self.threshold = threshold  
        self.left = left
```

```
self.right = right
self.value = value
```

```
class DecisionTreeScratch:
```

```
def __init__(self, max_depth=3):
    self.max_depth = max_depth
    self.root = None
```

```
def _gini(self, y):
    classes = np.unique(y)
    gini = 1.0
    for c in classes:
        p = np.sum(y == c) / len(y)
        gini -= p**2
    return gini
```

```
def _best_split(self, X, y):
    best_gini = float("inf")
    best_feature, best_threshold = None, None
    n_samples, n_features = X.shape
```

```
    for feature in range(n_features):
        thresholds = np.unique(X[:, feature])
        for t in thresholds:
            left_idx = X[:, feature] <= t
            right_idx = X[:, feature] > t

            if sum(left_idx) == 0 or sum(right_idx) == 0:
```

```
        continue
```

```
        gini_left = self._gini(y[left_idx])
```

```
        gini_right = self._gini(y[right_idx])
```

```
        weighted_gini = (
```

```
            sum(left_idx) * gini_left + sum(right_idx) * gini_right
```

```
        ) / n_samples
```

```
        if weighted_gini < best_gini:
```

```
            best_gini = weighted_gini
```

```
            best_feature = feature
```

```
            best_threshold = t
```

```
    return best_feature, best_threshold
```

```
def _build_tree(self, X, y, depth=0):
```

```
    if (
```

```
        depth >= self.max_depth
```

```
        or len(np.unique(y)) == 1
```

```
        or len(y) < 2
```

```
    ):
```

```
        leaf_value = (
```

```
            pd.Series(y).mode()[0] if len(y) > 0 else None
```

```
        )
```

```
        return Node(value=leaf_value)
```

```
    feature, threshold = self._best_split(X, y)
```

```
    if feature is None:
```

```
        leaf_value = pd.Series(y).mode()[0]
```

```
        return Node(value=leaf_value)
```

```
left_idx = X[:, feature] <= threshold
```

```
right_idx = X[:, feature] > threshold
```

```
left_child = self._build_tree(X[left_idx], y[left_idx], depth + 1)
```

```
right_child = self._build_tree(X[right_idx], y[right_idx], depth + 1)
```

```
return Node(
```

```
    feature=feature,
```

```
    threshold=threshold,
```

```
    left=left_child,
```

```
    right=right_child,
```

```
)
```

```
def fit(self, X, y):
```

```
    self.root = self._build_tree(X, y)
```

```
def _predict_sample(self, node, x):
```

```
    if node.value is not None:
```

```
        return node.value
```

```
    if x[node.feature] <= node.threshold:
```

```
        return self._predict_sample(node.left, x)
```

```
    return self._predict_sample(node.right, x)
```

```
def predict(self, X):
```

```
    return np.array([self._predict_sample(self.root, x) for x in X])
```

```
df = pd.read_csv("credit_score_data.csv")
```

```
X = df[["Income", "Age", "LoanAmount", "CreditHistory"]].values
```

```
y = df["CreditScore"].values
```

```
model = DecisionTreeScratch(max_depth=3)
```

```
model.fit(X, y)
```

```
predictions = model.predict(X)
```

```
accuracy = np.mean(predictions == y) * 100
```

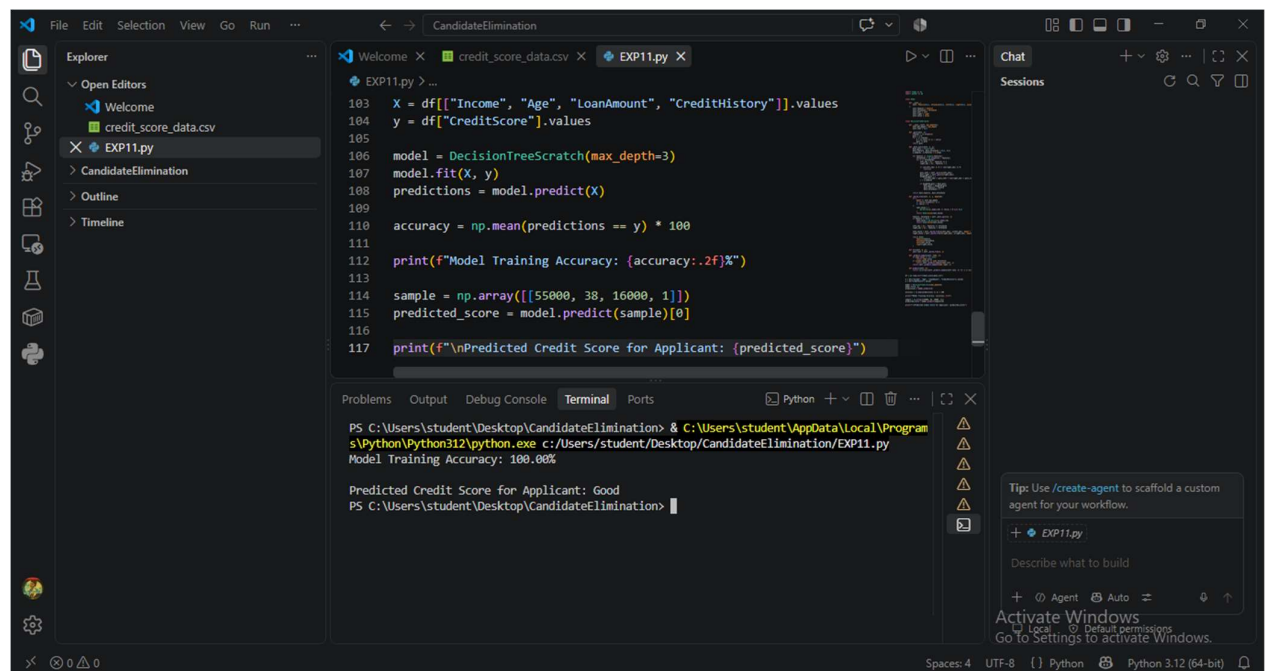
```
print(f"Model Training Accuracy: {accuracy:.2f}%")
```

```
sample = np.array([[55000, 38, 16000, 1]])
```

```
predicted_score = model.predict(sample)[0]
```

```
print(f"\nPredicted Credit Score for Applicant: {predicted_score}")
```

OUTPUT



The screenshot shows a Visual Studio Code editor window with a Python script named `EXP11.py` and its output in the terminal. The script defines a `DecisionTreeScratch` model with a maximum depth of 3, fits it to a dataset, and predicts the credit score for a specific applicant. The terminal output shows that the model training accuracy is 100.00% and the predicted credit score for the applicant is 'Good'.

```
File Edit Selection View Go Run ...
CandidateElimination
EXP11.py X
credit_score_data.csv X
EXP11.py X
103 X = df[["Income", "Age", "LoanAmount", "CreditHistory"]].values
104 y = df["CreditScore"].values
105
106 model = DecisionTreeScratch(max_depth=3)
107 model.fit(X, y)
108 predictions = model.predict(X)
109
110 accuracy = np.mean(predictions == y) * 100
111
112 print(f"Model Training Accuracy: {accuracy:.2f}%")
113
114 sample = np.array([[55000, 38, 16000, 1]])
115 predicted_score = model.predict(sample)[0]
116
117 print(f"\nPredicted Credit Score for Applicant: {predicted_score}")

Problems Output Debug Console Terminal Ports
PS C:\Users\student\Desktop\CandidateElimination> & C:\Users\student\AppData\Local\Programs\Python\Python312\python.exe c:/Users/student/Desktop/CandidateElimination/EXP11.py
Model Training Accuracy: 100.00%

Predicted Credit Score for Applicant: Good
PS C:\Users\student\Desktop\CandidateElimination>
```